Bincom Development Center

Cybersecurity Class

Assignment on

Network Attack Detection & Reporting

By

Ebere Anastasia Akpan

Date:
03/03/2026

1. Executive Summary

This project focuses on deploying a Cowrie SSH honeypot within a Dockerized environment to simulate malicious activity in a controlled lab setting. The primary goal was to replicate attacker behavior, capture authentication attempts, and analyze the resulting network traffic. Evidence collected during the simulation was then used to generate a structured incident report, documenting the detection of an SSH authentication attack and the investigative process.

2. Objective

The primary objectives are to:

- Build a honeypot, attack it, and capture the evidence.
- Detect and document unauthorized SSH authentication attempts.
- Assess the honeypot's effectiveness in capturing and reporting malicious activity.
- Provide actionable insights through a structured incident report.
- Simulate network reconnaissance and SSH brute-force attacks using Kali Linux, and to detect them using Cowrie Honeypot and Wireshark.

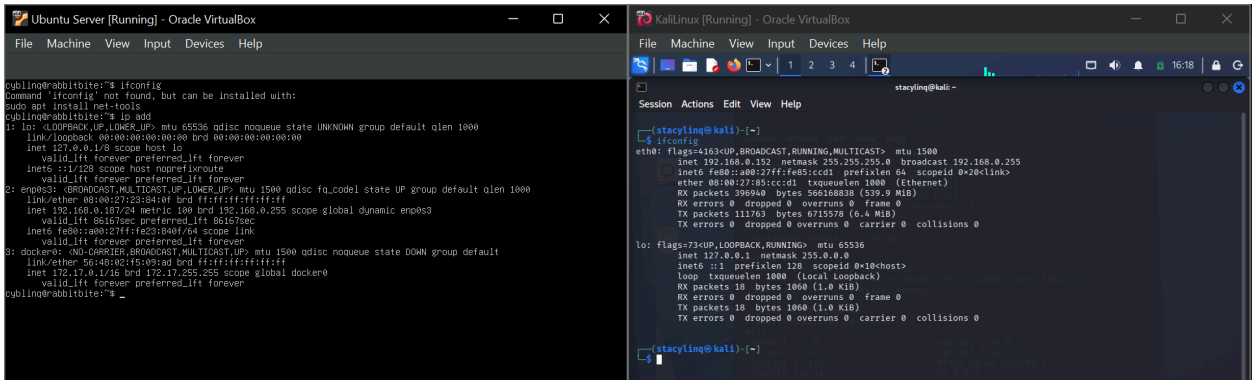
3. Environment Overview

Virtualization platform -	Oracle VirtualBox
Attacker Machine -	Kali Linux VM
Target Machine -	Ubuntu Server with Cowrie Honeypot
Network configuration -	Target Port: 2222 (SSH Honeypot), Protocol: TCP
SSH Monitoring -	Cowrie (Honeypot)
Reconnaissance -	Nmap
Packet Analysis -	Wireshark

4. Methodology

Step 1: Environment Setup

Virtual machines (2 VMs) were set up, the Bridge network was set to enable VMs communicate effectively but isolate them from the internet.



Deployment:

Cowrie was deployed inside a Docker container on Ubuntu Server:

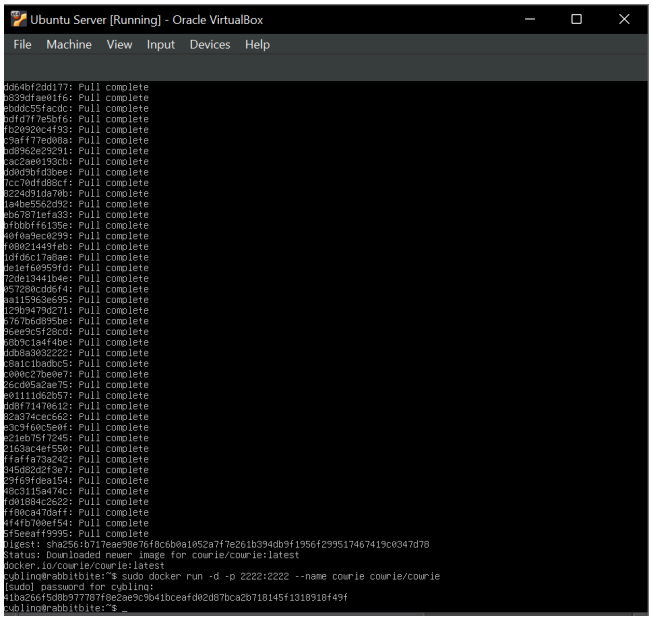
sudo apt install docker.io -y
sudo docker pull cowrie/cowrie
sudo docker run -d -p 2222:2222 --name cowrie cowrie/cowrie

This command deploys Cowrie, an SSH honeypot, in a Dockerized environment. Once running, it listens on port 2222 and records any SSH login attempts. This allows you to simulate attacker behavior, capture authentication attempts, and analyze logs/network traffic without exposing a real system to risk.

The container was verified using:

sudo docker ps

This exposed the SSH honeypot on TCP port 2222.



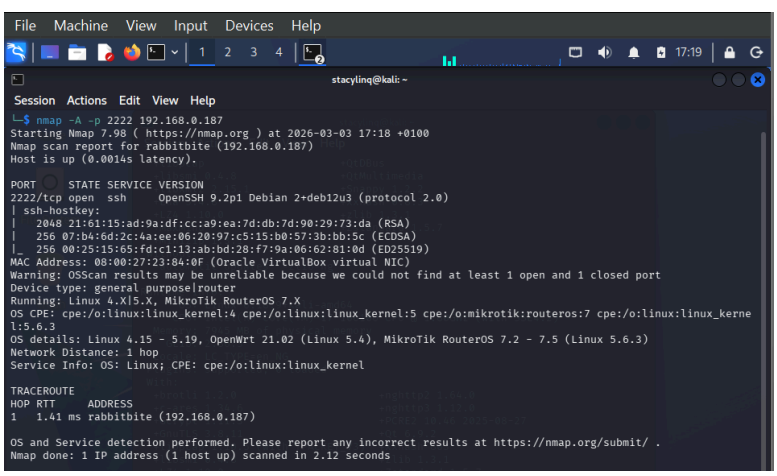
Step 2: Nmap Scanning (Basic, Version Detection, Aggressive Scan)

From Kali Linux:

nmap -A -p 2222 192.168.0.187

Result:

Port 2222 open
SSH service detected



Step 3: SSH Brute-force simulation

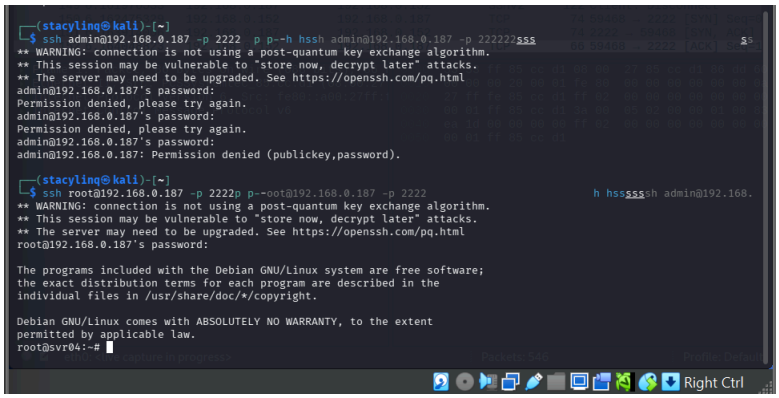
An SSH connection was initiated: (on Kali)

ssh root@192.168.0.187 -p 2222

This is an SSH client connection attempt to log in as the root user on the machine at IP 192.168.0.187, using SSH over port 2222. This is the port mapped to the Cowrie honeypot container.

Credentials used:

Username: root
Password: 242494



Step 4: Log Analysis from Cowrie: (on Ubuntu)

Authentication activity was extracted using:

sudo docker logs cowrie | grep "login"

Observed log entry:

login attempt [b'admin'/b'242494'] failed
login attempt [b'root'/b'242494'] succeeded (root is the username being used for login.)

```
cybling@rabbitbite:~$ sudo docker logs cowrie | grep "login"
[sudo] password for cybling:
/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:110: CryptographyDeprecationWarning: Trip
mat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in
  b"3des-cbc": (algorithms.TripleDES, 24, modes.CBC),
/cowrie/cowrie-env/lib/python3.13/site-packages/twisted/conch/ssh/transport.py:117: CryptographyDeprecationWarning: Trip
mat.decrepit.ciphers.algorithms.TripleDES and will be removed from cryptography.hazmat.primitives.ciphers.algorithms in
  b"3des-ctr": (algorithms.TripleDES, 24, modes.CTR),
2026-03-03T16:25:32+0000 [HoneyPotSSHTransport,27,192.168.0.152] login attempt [b'root'/b'suCCess29'] succeeded
2026-03-03T16:32:23+0000 [HoneyPotSSHTransport,28,192.168.0.152] login attempt [b'admin'/b'242494'] failed
2026-03-03T16:32:24+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:32:44+0000 [HoneyPotSSHTransport,28,192.168.0.152] login attempt [b'admin'/b'242494'] failed
2026-03-03T16:32:45+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:32:53+0000 [HoneyPotSSHTransport,28,192.168.0.152] login attempt [b'admin'/b'2ba2Ra2'] failed
2026-03-03T16:32:54+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:34:53+0000 [HoneyPotSSHTransport,29,192.168.0.152] login attempt [b'admin'/b'242494'] failed
2026-03-03T16:34:54+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:35:06+0000 [HoneyPotSSHTransport,29,192.168.0.152] login attempt [b'admin'/b'2ba2ra2'] failed
2026-03-03T16:35:07+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:35:18+0000 [HoneyPotSSHTransport,29,192.168.0.152] login attempt [b'admin'/b'2ba5Ra2'] failed
2026-03-03T16:35:19+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T16:35:44+0000 [HoneyPotSSHTransport,30,192.168.0.152] login attempt [b'root'/b'2ba2Ra2'] succeeded
2026-03-03T16:49:36+0000 [HoneyPotSSHTransport,31,192.168.0.152] login attempt [b'root'/b'242494'] succeeded
2026-03-03T17:10:12+0000 [HoneyPotSSHTransport,50,192.168.0.152] login attempt [b'admin'/b'242494'] failed
2026-03-03T17:10:13+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T17:10:23+0000 [HoneyPotSSHTransport,50,192.168.0.152] login attempt [b'admin'/b'242494'] failed
2026-03-03T17:10:24+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T17:10:50+0000 [HoneyPotSSHTransport,50,192.168.0.152] login attempt [b'admin'/b'jumps'] failed
2026-03-03T17:10:52+0000 [cowrie.ssh.userauth.HoneyPotSSHUserAuthServer#debug] unauthorized login: ()
2026-03-03T17:13:12+0000 [HoneyPotSSHTransport,51,192.168.0.152] login attempt [b'root'/b'242494'] succeeded
cybling@rabbitbite:~$ _
```

Log Analysis Summary:
Cowrie captured: Source IP address (192.168.0.152), Username/password combinations attempted (admin,root/242494), Session timestamps (2026-03-03T17:13:12+0000).
Commands executed by attacker Indicators of Compromise (IOCs): Repeated authentication failures and High-frequency connection attempts

Key Notes:

- Cowrie is designed to never grant real access. Even if you type a “valid” username/password, it only simulates a shell environment and records the attempt.
- Every login attempt (username + password combination) is captured in Cowrie’s logs. This is how you generate evidence of brute-force or credential-guessing attacks.
- One can use this to simulate attacker behavior by trying multiple combinations, which will later appear in the log files for analysis.

Step 5: Packet capture using Wireshark

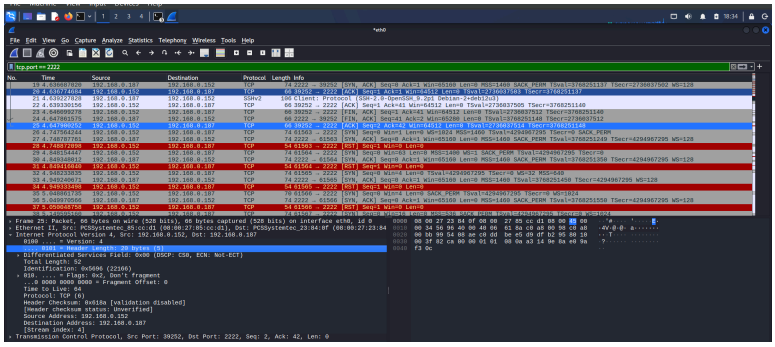
Network Traffic Analysis- Wireshark was used to capture the attack traffic.

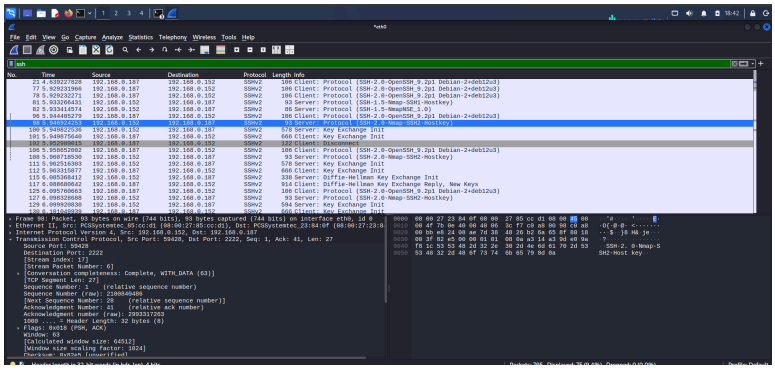
Filter applied:

tcp.port == 2222
SSH

Captured evidence included:

TCP three-way handshake
SSH key exchange
Authentication negotiation





4. Incident Report

Incident Type:

Unauthorized SSH Authentication Attempt

Detection Method:

Honeypot logging and packet capture analysis

Findings & Observations:

- Open ports identified
- Services detected
- Login attempts captured by Cowrie
- Packet-level evidence observed in Wireshark
- Indicators of reconnaissance and brute-force activity

Deliverables:

(Attach Screenshots) Include:

- Nmap scan output screenshots
- Cowrie log extracts

- Wireshark capture screenshots

Impact Assessment:

The simulated attack did not result in any real compromise because it was conducted in an isolated honeypot environment. However, if similar activity were observed in a production system, it could indicate:

- An active brute-force attack against SSH services
- Credential harvesting attempts
- Unauthorized remote access attempts
- Early-stage reconnaissance preceding further exploitation

Risk Level (Production Scenario): Medium

Such activity could lead to account compromise, lateral movement, or full system intrusion if not promptly mitigated.

5. Recommendations

To reduce the risk of SSH-based attacks in production environments, the following controls are recommended:

- Implement strong password policies
- Enable firewall restrictions
- Configure intrusion detection systems (IDS)
- Change default ports and disable unused services
- Enable account lockout policies
- Continuously monitor authentication and system logs for anomalies.

6. Conclusion

This project demonstrates practical detection and analysis of SSH authentication attacks within a controlled lab environment using a Docker-based honeypot. The exercise reinforces the importance of monitoring exposed services, identifying malicious patterns early, and implementing layered security controls to protect production systems.